LUMEN — Crime Intelligence & Visualization Platform

Complete Architecture & System Documentation

Document Structure

- **VOLUME 1: ARCHITECTURE DESIGN**
  - Executive Summary
  - Architecture Principles
  - MVP Architecture
  - Detailed Component Design
  - Data Flow Architecture
  - Database Domain Architecture
  - API Architecture
- **VOLUME 2: SCALABILITY & CLOUD ARCHITECTURE**
  - Catalyst MVP Deployment
  - AWS Production Architecture
  - Kubernetes Migration Strategy
  - Multi-State Expansion
  - Capacity Planning
  - Cost Modeling
  - Disaster Recovery
  - Security Architecture
- **VOLUME 3: JUDGE DOCUMENTATION & DECISION ANALYSIS**
  - Architecture Justifications
  - Architectural Decision Records (ADRs)
  - Tradeoff Analysis
  - Future Scope
  - Risk Assessment
  - Technical Debt Register
  - Presentation Talking Points
  - Judge Q&A Cheat Sheet

VOLUME 1 — ARCHITECTURE DESIGN

1. Executive Summary

1.1 Project Overview

Crime Intelligence & Visualization Platform designed to revolutionize traditional crime analytics for the Karnataka State Police (KSP). The platform is architected to modernize crime analysis by transforming fragmented, disparate crime records into an integrated, centralized, and intelligent operational ecosystem.

By fusing predictive analytics, geospatial intelligence, criminal network modeling, real-time anomaly detection, and interactive dashboarding, LUMEN empowers law enforcement agencies to transition from reactive policing to proactive, data-driven crime prevention.

Key Differentiator: Unlike legacy police database applications and Record Management Systems (RMS) that serve merely as static storage vaults and manual PDF reporting portals, **LUMEN is built as an active intelligence generation engine**. It operates continuously on ingestion pipelines to uncover hidden structural insights automatically.

1.2 Business Objectives

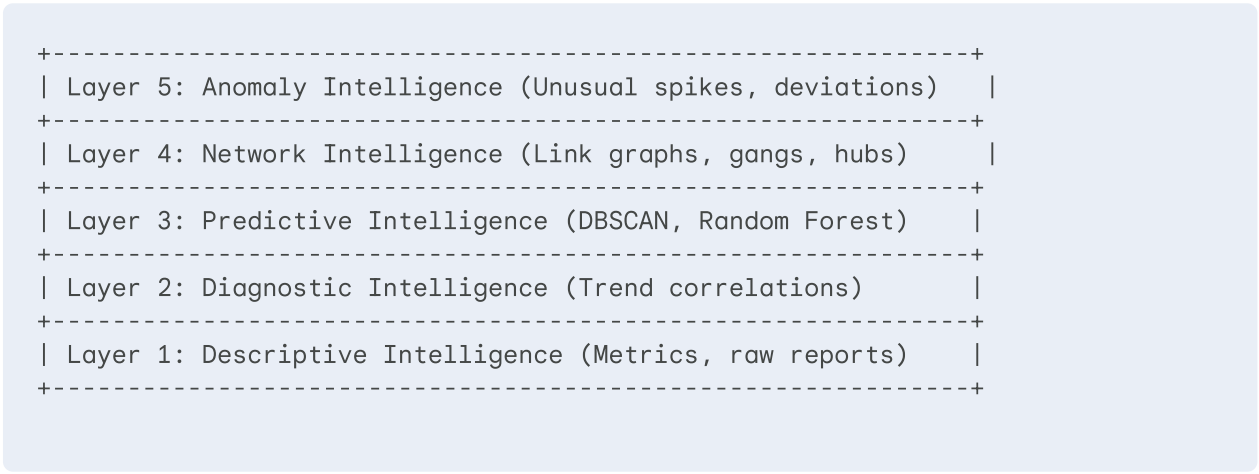
Primary Objectives

1. **Centralized Crime Intelligence:** Create a unified semantic layer for statewide crime data integration and intelligent querying.
2. **Predictive Policing:** Forecast emerging high-risk patrol zones and geographic crime trends before occurrences.
3. **Criminal Network Discovery:** Automatically uncover implicit relationships, operational dependencies, and organizational structures among repeat suspects, gangs, and syndicates.
4. **Real-Time Visual Analytics:** Supply high-fidelity interactive map layers, heatmaps, temporal charts, and operational metrics.
5. **Investigation Acceleration:** Minimize manual file parsing and suspect correlation times through automated link-analysis and anomaly indicators.
6. **Resource Allocation Optimization:** Assist command structures in matching patrolling rosters to dynamic high-risk zones.

Success Metrics

Metric	Current State	Target
Crime Analysis Time	Hours to Days of manual spreadsheet collation	Under 3 Minutes via automated query dashboards
Pattern Detection	Relies on manual memory and physical case folders	Automated anomaly and correlation analysis
Crime Forecasting	Non-existent or based on basic seasonal assumptions	ML-Based spatial and temporal probability mapping
Criminal Relationship Discovery	Manual link charts drawn post-interrogation	Automatic graph generation based on shared vectors
Statewide Visibility	Fragmented access, isolated district reports	Unified, role-restricted dashboard view

LUMEN structures its computational analytics across **five layered intelligence tiers**:



Layer 1 — Descriptive Intelligence

- **Core Question:** *What happened, when, and where?*
- **Analytical Operations:** * Real-time geospatial plotting of historical records.
  - Multi-dimensional filtering by district, station, date, and crime category (IPC/SLL).
  - Rolling KPI card generation representing cumulative counts, percentage shifts, and category breakdowns.

Layer 2 — Diagnostic Intelligence

- **Core Question:** *Why did it occur in this manner?*
- **Analytical Operations:**
  - Spatial correlation analysis against demographic and urban attributes.
  - Temporal breakdown modeling (day-of-week, diurnal cycles, seasonal deviations).
  - Crime progression matrices showing transition trajectories of repeat offenders.

Layer 3 — Predictive Intelligence

- **Core Question:** *Where and when is a crime highly likely to happen next?*
- **Analytical Operations:**
  - Geospatial micro-hotspot forecasting utilizing geographic clustering algorithms.
  - Predictive danger-index modeling scoring police jurisdiction subdivisions for patrolling priority.
  - Categorical probability projections utilizing ensemble tree classification approaches.

Layer 4 — Network Intelligence

- **Core Question:** *Who is connected, how are they organized, and who holds influence?*
- **Analytical Operations:**
  - Multi-modal network configuration tracking (Suspect-Suspect, Suspect-Crime, Suspect-Location, Suspect-Vehicle).
  - Centrality computation (Degree, Betweenness, Eigenvector) mapping critical transaction nodes and gang ringleaders.

Layer 5 — Anomaly Intelligence

- **Core Question:** *What patterns deviate abnormally from historical baselines?*
- **Analytical Operations:**
  - Automated time-series anomaly flags detecting unexpected volumetric crime spikes in localized areas.
  - Behavioral outlier detection highlighting anomalous modus operandi (MO) signatures.
  - Early-warning event monitoring for systemic escalations.

2. Architecture Principles

2.1 MVP First

Hackathons reward high utility, runtime stability, clear separation of concerns, and demonstrable features. Overengineering infrastructure early introduces point-of-failures, slows implementation cycles, and obscures direct value.

Therefore, our primary implementation rule is: **Monolith First → Services Later**. By executing a modular monolithic design patterns inside the backend, we preserve the rapid compilation, shared runtime safety, and instantaneous execution of a single application deployment while enforcing clean functional domain isolation.

2.2 Intelligence-Centric Design

In legacy database systems, the flow is structurally direct and computationally static:

```
Raw Data ---> Database Store ---> Basic Query Read
```

LUMEN modifies this architecture to make analytics the core operating engine of the system. Data is transformed systematically into structural insights:

```
Raw Data ---> Relational Storage ---> Analytical Enrichment Pipeline ---> Structu
```

Every user interface asset, reporting model, and database trigger is purpose-built to extract, format, display, and validate predictive or relational insights.

2.3 Modular Architecture

Each analytics and functional system is completely containerized within the codebase, communicating via well-defined operational APIs. The modules are:

- **Dashboard Engine:** Manages aggregation query profiles and performance-optimized read layers.
- **Geospatial Hotspot Engine:** Fuses geographic coordinates into density cluster profiles.
- **Predictive ML Engine:** Performs model inference, feature preparation, and parameter evaluation.

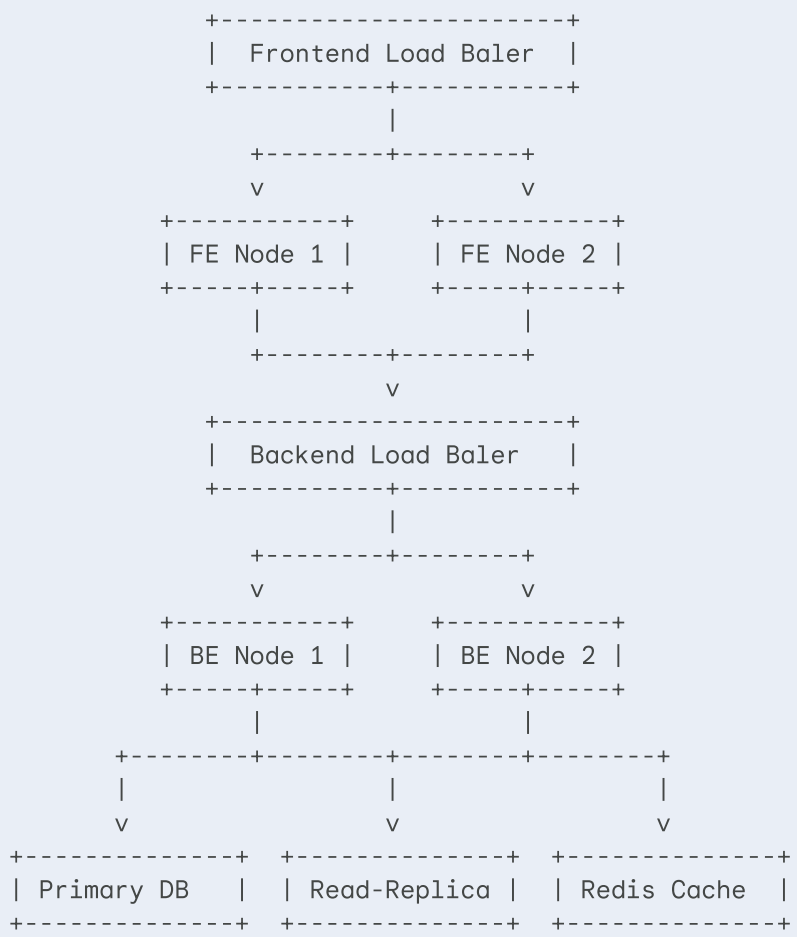
matrices.

- **Anomaly Alerting Engine:** Tracks sliding window historical metrics to flag outliers.

This guarantees that the underlying machine learning models or graphing frameworks can be refactored, retrained, or swapped out entirely without mutating the user interface or relational schemas.

2.4 Horizontal Scalability

Every system layer is decoupled to allow isolated horizontal scaling.

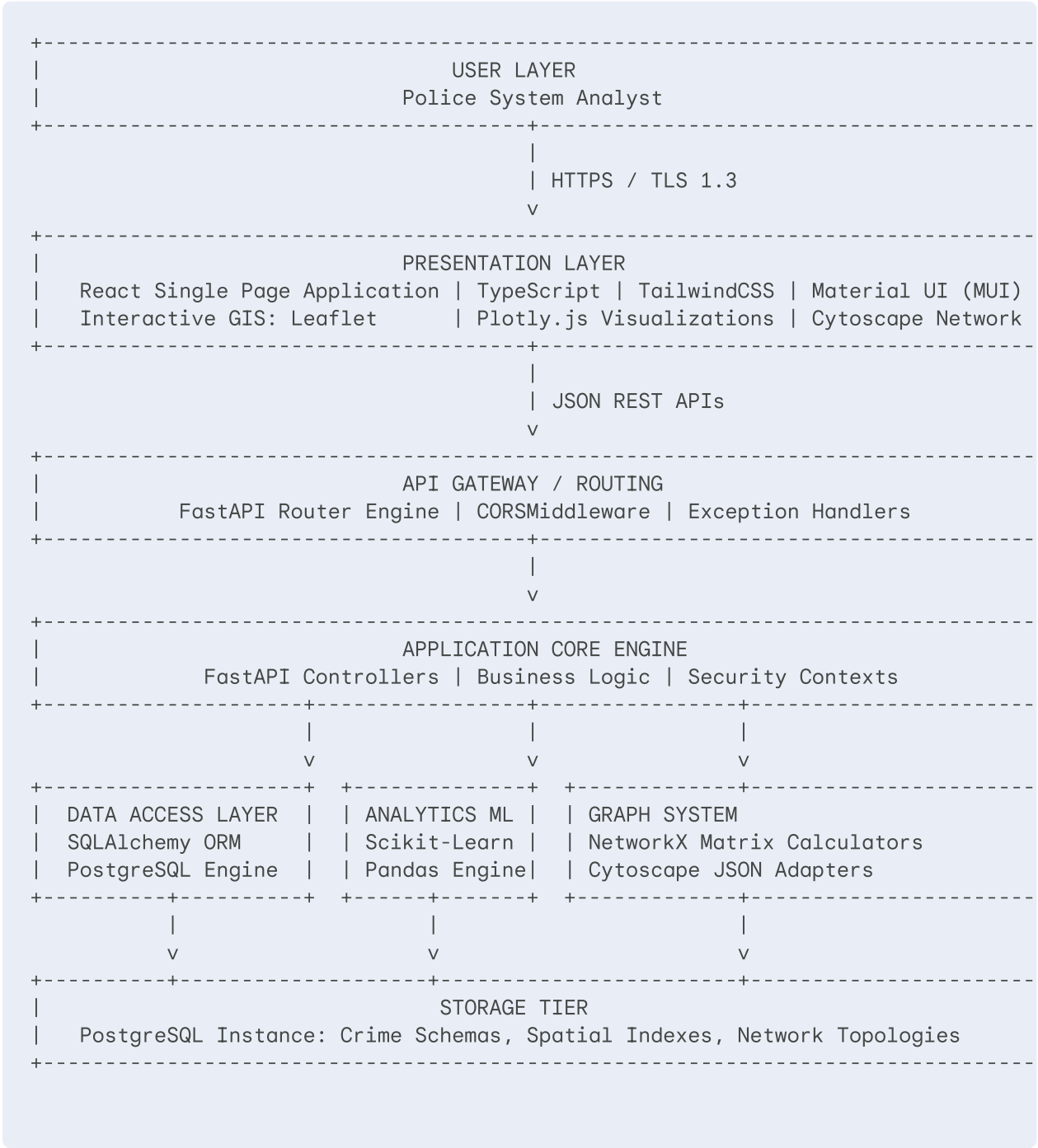


Component	Scaling Strategy	Scaling Path
Frontend UI	Static Assets, CDN Distribution	1 instance → N Global Edge Nodes
Backend API	Stateless Application Instances	1 FastAPI worker → N Load-Balanced Container Instances
Database	Relational Clustering, Read Replication	Single Instance Write Master → Multi-AZ Read Replicas
Analytics Core	Memory-Optimized Compute Instances	Single-threaded execution → Distributed Task Workers (Celery)

While optimized for rapid execution within Zoho Catalyst's developer ecosystem for the hackathon, the underlying codebases and storage schemas rely strictly on open-source standards (React, Python FastAPI, PostgreSQL). No proprietary lock-in mechanisms are utilized, enabling immediate containerization and deployment onto AWS, Azure, GCP, or secure on-premise government private clouds.

3. MVP Architecture

3.1 High-Level Architecture Diagram



3.2 Frontend Layer

Frontend Technologies

- **React Engine:** Direct state management and component rendering.

models and API models.

- **Material UI (MUI):** Enterprise UI layouts, ensuring professional government aesthetics, data grids, modals, and charts.

Core Responsibilities

- Client-side application routing and UI page transitions.
- Form validation, search configurations, filtering bindings, and analytical views.
- GIS integration rendering geospatial elements, district boundaries, and cluster overlays.
- Interactive network node mapping rendering, dynamic link tracking, and layout manipulation.

Frontend Constraints

- **Pure Presentation Policy:** Absolutely zero business logic, raw coordinate clustering calculations, or model parameter optimization is done on the client. The frontend is strictly a visualization and interactions port.

3.3 Backend Layer

Backend Technologies

- **FastAPI Framework:** ASGI based Python engine built for performance and asynchronous data pathways.
- **SQLAlchemy:** Object Relational Mapper interfacing with the storage layer without manual SQL repetition.
- **Pydantic:** Payload serialization, automatic validation, and API schema definitions.

Core Responsibilities

- Secure routing, execution monitoring, payload verification, and error parsing.
- Dynamic analytical query processing against PostgreSQL.
- ML model orchestration executing automated inference.
- Graph processing converting raw database structures to layout matrices.

3.4 Database Layer

Database Technologies

- **PostgreSQL:** Relational core database handling strict transactional integrity, indexing, complex views, and recursive relationship processing.

Structural Holdings

- Complete normalized relational schemas spanning case profiles, suspects, geographic administrative limits, model outputs, and auditing trails.

3.5 Intelligence Layer

Core Frameworks

classification systems.

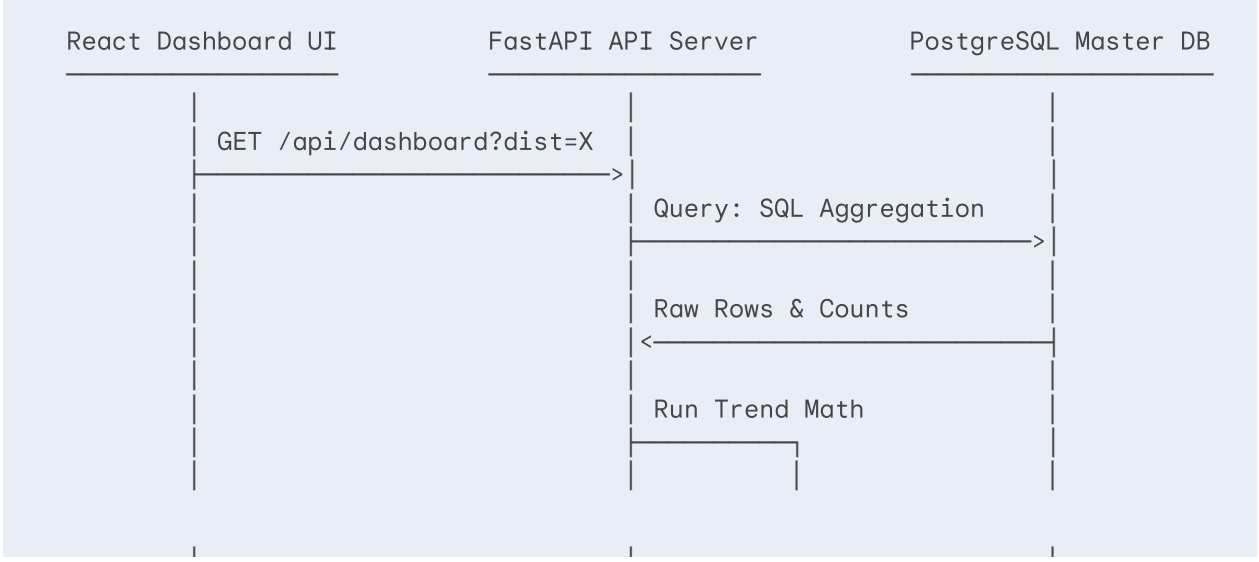
- **NetworkX:** Compute network properties, link layouts, paths, communities, and centrality maps.
- **Pandas & NumPy:** Linear algebra transformations and performance data processing matrices.

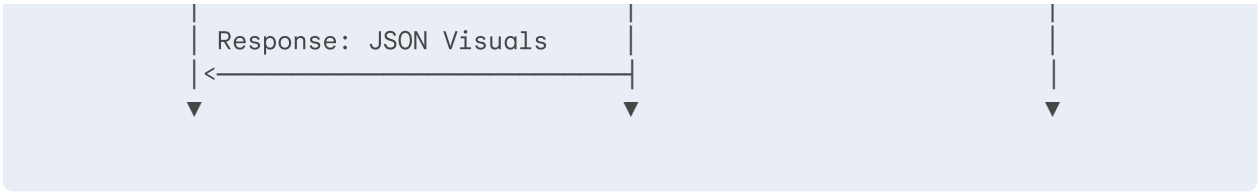
4. Detailed Component Design

```
+-------------------------------------------------------------------------------
|                                                LUMEN COMPONENT SCHEMATIC
+-------------------------------------------------------------------------------
|  [CRIME DASHBOARD MODULE]
|  - Inputs: Raw Crime Records, Time Series Parameters, Spatial Boundaries
|  - Processing: SQL Aggregations, Categorical Grouping, Trend Slope Scoring
|  - Outputs: Target KPI Targets, Distribution Tables, Plotly Chart Datasets
+-------------------------------------------------------------------------------
|  [GEOSPATIAL HEATMAP ENGINE]
|  - Inputs: Latitude, Longitude, Date Constraints, Category Filters
|  - Processing: DBSCAN Density-Based Spatial Clustering of Applications with Noi
|  - Outputs: Coordinate Bounds, Cluster Weights, Noise Classifications
+-------------------------------------------------------------------------------
|  [PREDICTIVE MACHINE LEARNING ENGINE]
|  - Inputs: Historical Spatiotemporal Vectors, Climatic & Socioeconomic Baseline
|  - Processing: Random Forest Classification, Gradient Boosting Regression, Feat
|  - Outputs: Zone-Based Danger Probabilities, Feature Importance Weightings
+-------------------------------------------------------------------------------
|  [CRIMINAL NETWORK GRAPH ENGINE]
|  - Inputs: Suspect Profiles, Joint Incident Records, Association Types
|  - Processing: Girvan-Newman Communities, Degree Centrality Metrics, Betweennes
|  - Outputs: Cytoscape Nodes (Suspects/Crimes), Cytoscape Edges (Connections), C
+-------------------------------------------------------------------------------
|  [ANOMALY MONITORING ENGINE]
|  - Inputs: Rolling Volume Metrics, Time Intervals, Category Baselines
|  - Processing: Isolation Forest Algorithms, Standard Deviation Outlier Flags
|  - Outputs: Anomaly Alerts, Spike Intensity Metrics, Divergence Reports
+-------------------------------------------------------------------------------
```

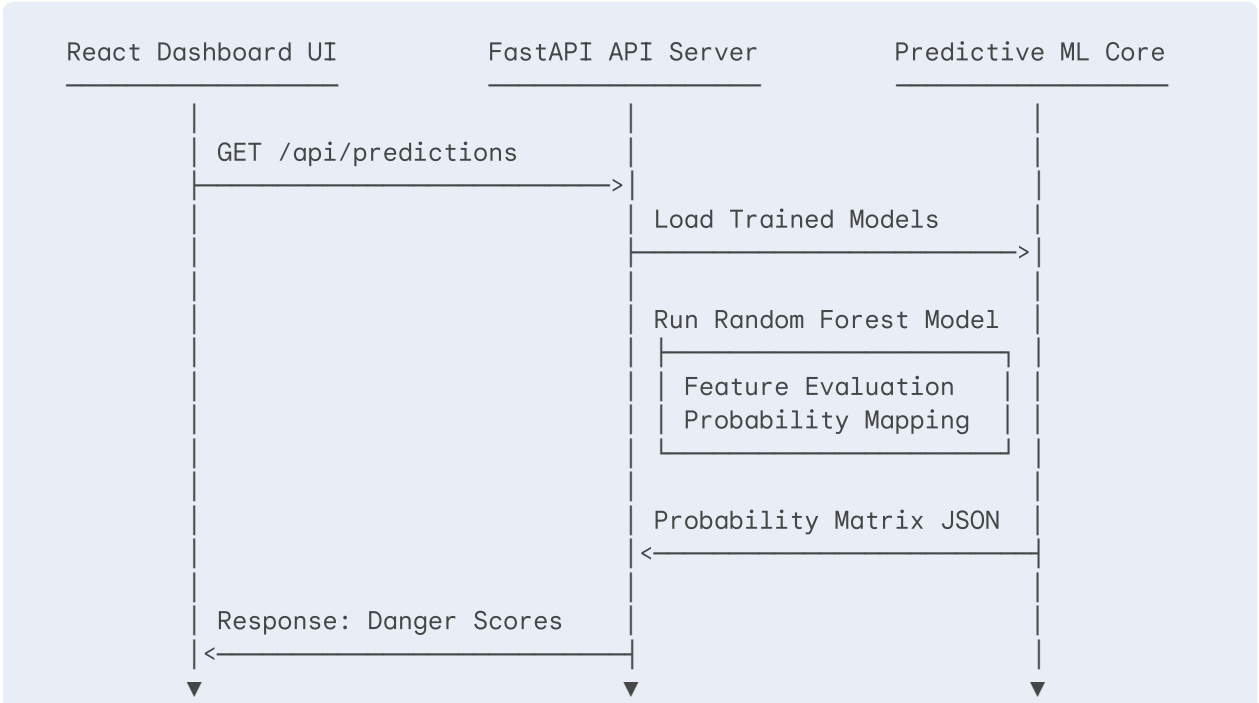
5. Data Flow Architecture

5.1 Dashboard Data Flow

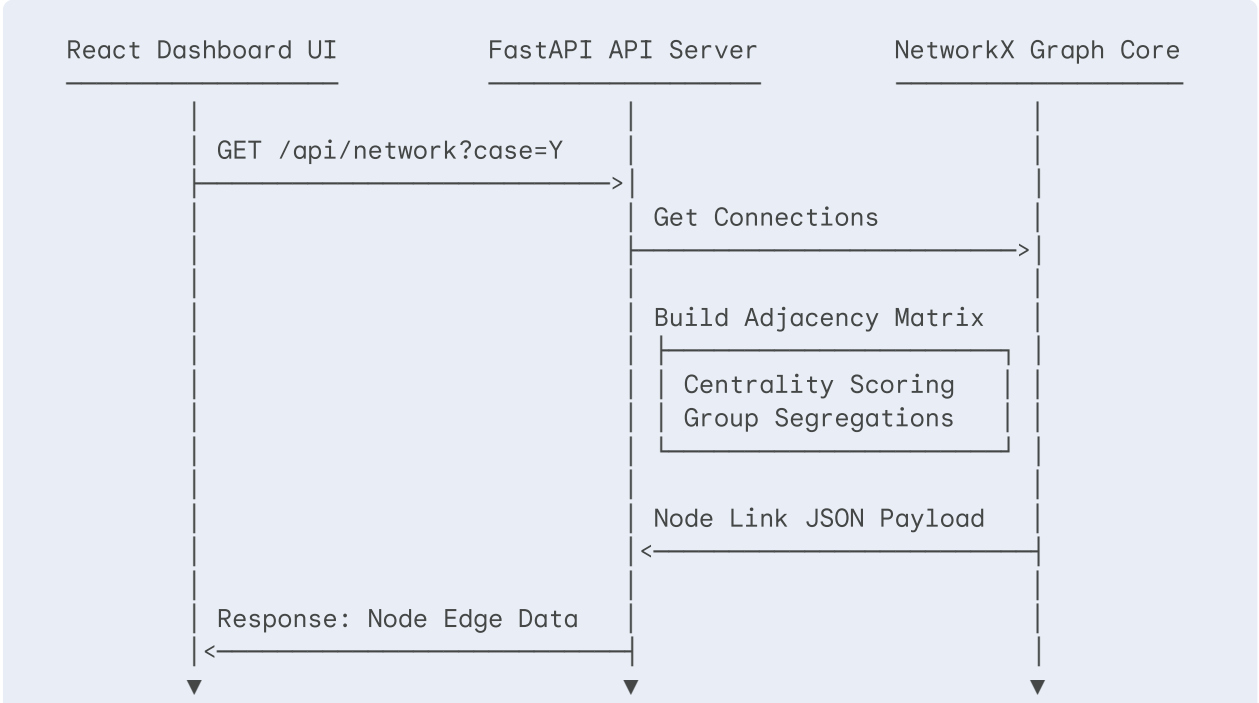




5.2 Predictive Analytics Flow

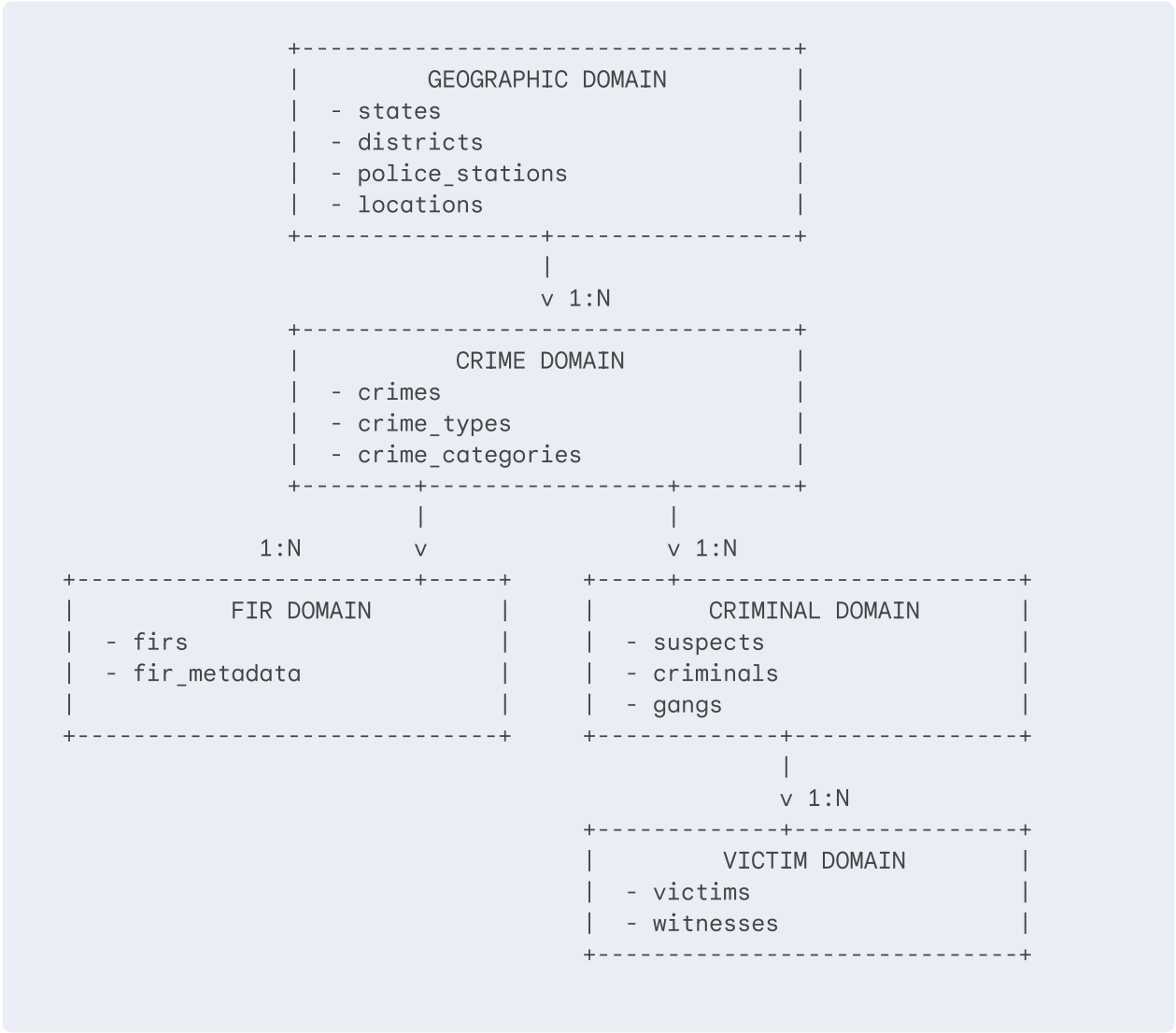


5.3 Network Analytics Flow



6. Database Domain Architecture

normalized schemas containing structural domains:



- **Geographic Domain:** Represents administrative boundaries.
- **Crime Domain:** Main data records with physical details.
- **FIR Domain:** Unstructured police reports and extraction logs.
- **Criminal Domain:** Personal profile histories, alias tracking, and organization definitions.
- **Victim Domain:** Details concerning witnesses and complainants.
- **Intelligence Domain:** Output structures for predictions, alerts, and calculated parameters.
- **Network Domain:** Pre-computed relationship connections and topological structures.

7. Database Entity Count

Implementation State	Targeted Table Count	Architectural Strategy
MVP Stage	**15 - 20 Tables**	Normalized core tables tracking critical crime, suspect, and spatial features.

		and station logistics.
State Scale	**60 - 80 Tables**	Addition of audit tracking, telemetry logs, configuration panels, and ingestion archives.
National Scale	**100+ Tables**	Cross-jurisdictional synchronization logs, legacy translation matrix mapping, federated registry layers.

8. API Architecture

All endpoints accept and return `application/json` structures.

Dashboard APIs

- `GET /api/dashboard`
  - **Description:** Retrieves cumulative KPIs, category divisions, and time-series series metrics.
- `GET /api/trends`
  - **Description:** Provides directional trending slopes, seasonal variance calculations, and category comparisons.
- `GET /api/districts`
  - **Description:** Returns comparative metrics scored by geographic districts.

Heatmap APIs

- `GET /api/hotspots`
  - **Description:** Processes spatiotemporal filters to return DBSCAN coordinated cluster clusters.
- `GET /api/geospatial`
  - **Description:** Supplies polygon coordinates and geographic metadata for district boundaries.

Prediction APIs

- `GET /api/predictions`
  - **Description:** Provides localized geographic probability mapping based on the historical baseline.
- `GET /api/risk-scores`
  - **Description:** Evaluates administrative subdivisions to issue dynamic tactical safety indexes.

Network APIs

- `GET /api/network`
  - **Description:** Extracts connection structures from case associations to return Cytoscape-formatted JSON.
- `GET /api/suspects/{id}`
  - **Description:** Displays unique suspect profiling records including network graphs and risk indices.

- `GET /api/anomalies`
  - **Description:** Isolates and extracts sudden spatiotemporal activity surges based on isolation forest passes.
- `GET /api/alerts`
  - **Description:** Serves a list of active operational warnings to coordinate emergency responses.

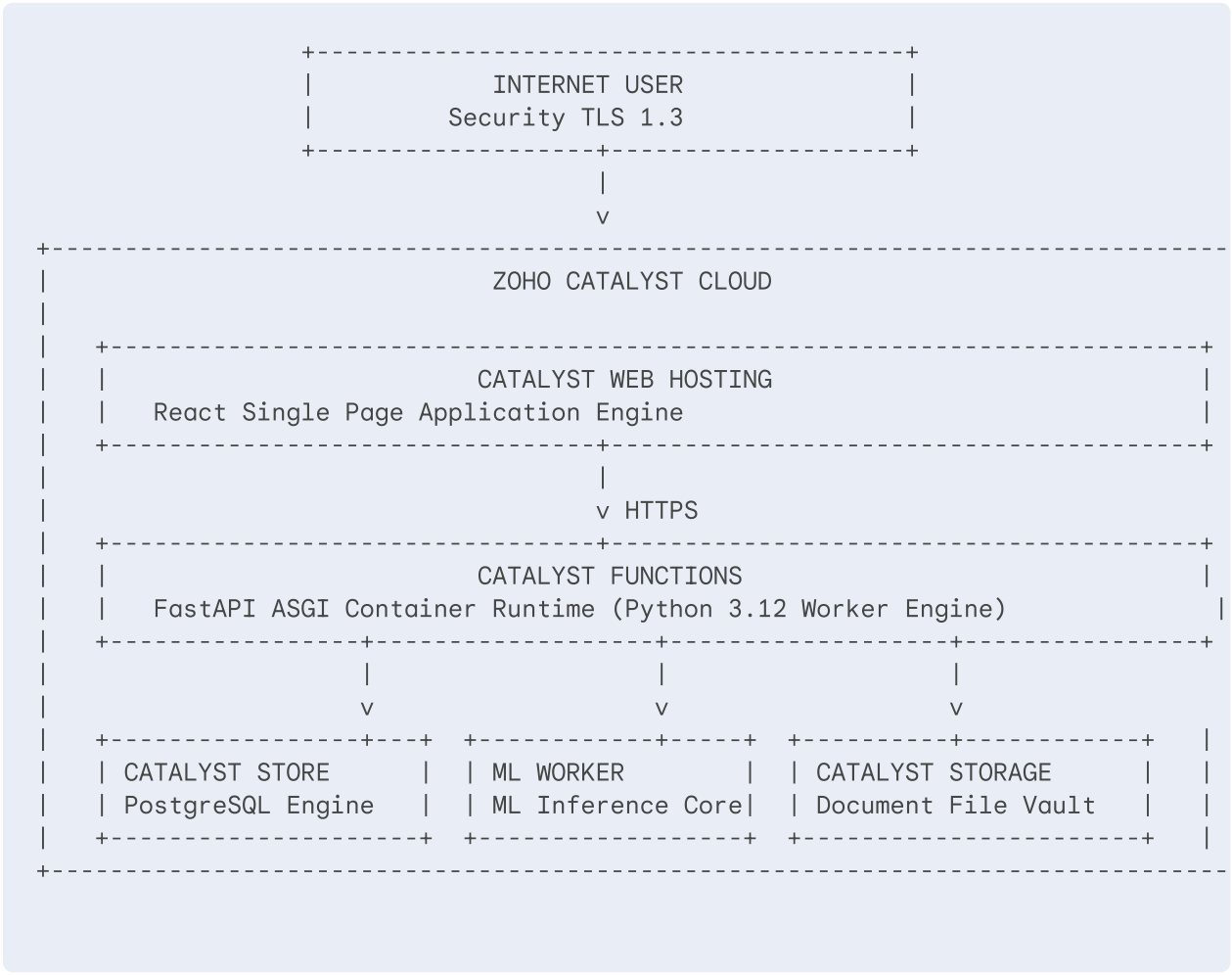
VOLUME 2 — SCALABILITY & CLOUD ARCHITECTURE

9. Catalyst MVP Deployment Architecture

9.1 Purpose

For MVP delivery, the system must deploy onto **Zoho Catalyst**. Zoho Catalyst provides an integrated serverless architecture, which guarantees zero active infrastructure management overhead, fast compilation cycles, automated SSL provisioning, and auto-scaling logic matching developer requirements.

9.2 MVP Deployment Topology



9.3 Catalyst Service Mapping

Infrastructure Core Element	Corresponding Zoho Catalyst Deployment System

API Backend Routing	Catalyst Functions (Python Advanced Container execution)
Relational Storage	Catalyst Data Store (PostgreSQL compatible engines)
Logging Hub	Catalyst Logs (Aggregated execution output capture)
File Storage Vault	Catalyst Object Storage (Secure digital file system storage)
Operational Telemetry	Catalyst Monitoring (Performance execution logs)

9.4 MVP Resource Assumptions

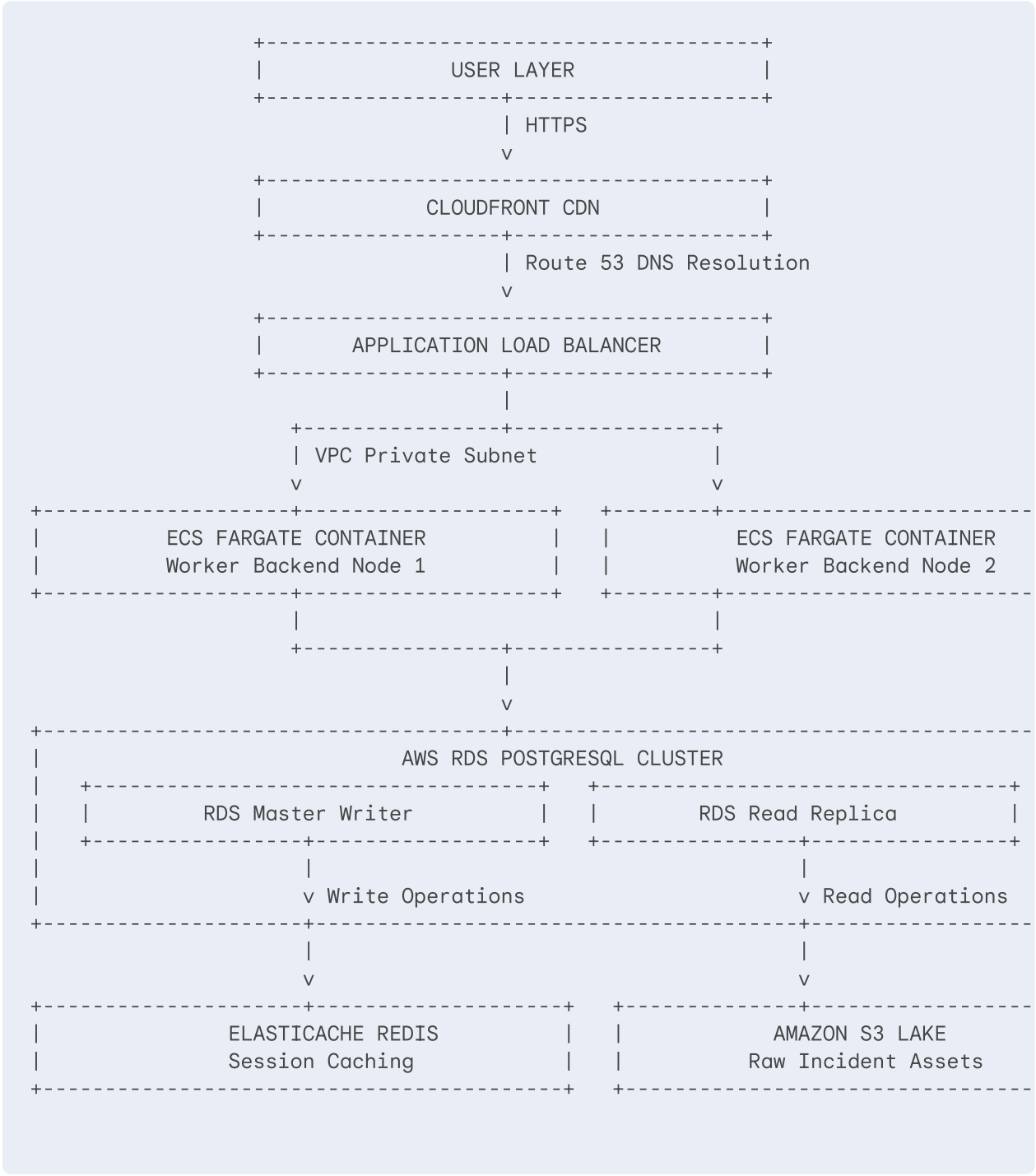
Parameter	MVP Value Allocation
Concurrent Active Sessions	10 to 25 active police analyst profiles
Active Case Load (Seed DB)	50,000 to 100,000 unified historical records
API Ingestion Volume	2,000 to 5,000 transactions daily
Model Inference Invocation	100 to 200 spatial-temporal updates daily
Storage Capacity Needs	Less than 1 Gigabyte relational database footings

9.5 MVP Cost Estimation

Operational Resource Element	Monthly Cash Allocation
React UI Static Hosting	₹0 (Catalyst Developer Ecosystem)
FastAPI Backend Executions	₹0 (Catalyst Developer Ecosystem)
PostgreSQL Base Engines	₹0 (Catalyst Developer Ecosystem)
ML Engine Orchestration	₹0 (Local inline CPU execution workers)
Catalyst Core Free Tiers	₹0 (Developer sandbox accounts)
Relational Storage Footprint	₹0 (Under 1GB free thresholds)
Cumulative MVP Expense	**₹0 to ₹500/Month** (Hobby Scale Limits)

10. AWS Production Architecture

To host Karnataka-scale databases containing millions of records securely, the topology scales to **Amazon Web Services (AWS)** using a private government cloud.



10.2 Why ECS First?

Many developer projects integrate Kubernetes prematurely. While Kubernetes (EKS) represents the industry standard for microservices orchestration, deploying it for an MVP or medium-scale pilot introduces complex networking, security configurations, and operational cost.

We choose **Amazon ECS with Fargate** because:

- **Zero Host Management:** Fargate eliminates manual VM provisioning, scaling, patching, and operational maintenance.
- **Cost Efficiency:** Billing runs strictly on CPU/Memory usage per second, preventing high baseline cluster costs.

to each container, locking database access at the OS level.

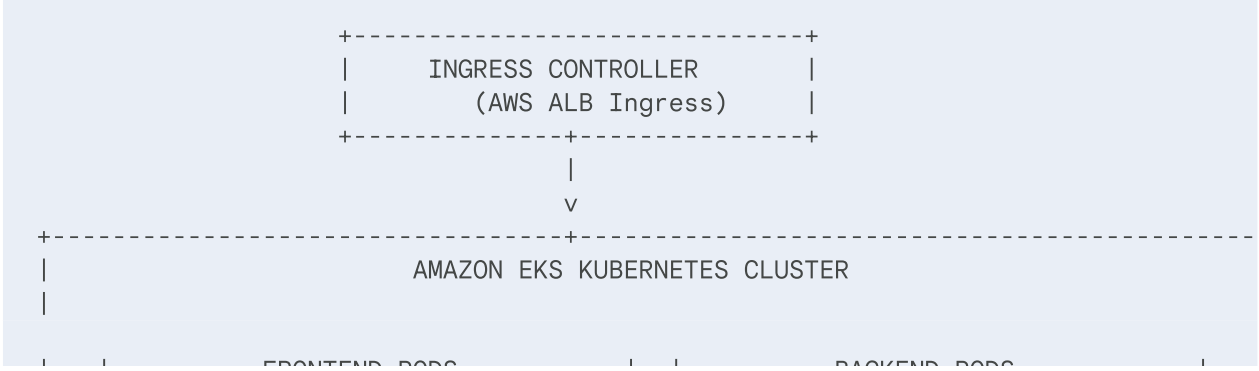
- **Rapid CI/CD Integration:** Integrates into GitHub Actions without custom helm-chart development.

10.3 AWS Service Selection

Operational Segment	Chosen AWS Service	Structural Justification
DNS Core	Amazon Route 53	High-availability domain routing with integrated health verification checks.
CDN Network	Amazon CloudFront	Delivers front-end static compiled assets with high speed.
Load Balancer	AWS ALB	Dispatches API transactions across active server containers.
Compute Engine	AWS ECS on Fargate	Serverless container execution keeping application instances stateless and safe.
Relational Core	Amazon RDS PostgreSQL	Managed multi-AZ high availability clusters with automated encryption.
Caching Tier	Elasticache for Redis	High-speed, in-memory caching to reduce core database load.
Object System	Amazon S3	Secure storage for structured case evidence and raw archives.
Auditing & Logs	Amazon CloudWatch	Captures application telemetry and alerts on operational errors.
Network Guard	AWS IAM / KMS	Implements strict RBAC and manages cryptographic keys.
Secrets Engine	AWS Secrets Manager	Isolates credentials away from build-pipelines and application code.

11. Kubernetes Migration Strategy

As LUMEN scales to support multiple states, the platform migrates to **Kubernetes (EKS)** to manage distributed, microservices-based containers across geographical regions.



```
|   |          FRONTEND PODS            |  |          BACKEND PODS             |
|   |   HPA Auto Scaling                |  |   HPA Auto Scaling                |
|   +-----------------------------------+  +-----------------------------------+
|   +-----------------------------------+  +-----------------------------------+
|   |         PREDICTION PODS           |  |           NETWORK PODS            |
|   |   Independent Inference Units     |  |   Graph Processing Containers     |
|   +-----------------------------------+  +-----------------------------------+
|   +-----------------------------------+  +-----------------------------------+
|   |          ANALYTICS PODS           |  |           ALERT PODS              |
|   |   Distributed pandas workers      |  |   Event Stream Outlier Flags      |
|   +-----------------------------------+  +-----------------------------------+
+-----------------------------------+-------------------------------------------
                                    |
                                    v
                      +--------------+---------------+
                      |     DATABASE LAYER CLUSTER    |
                      |   CockroachDB / Aurora Multi  |
                      +-------------------------------+
```

11.1 Migration Stages

```
STAGE 1 (Hackathon): Zoho Catalyst Serverless
                      |
                      ▼
STAGE 2 (Pilot): AWS ECS Single-AZ Container Instance
                      |
                      ▼
STAGE 3 (State Scale): AWS ECS Multi-AZ Auto-Scaling Cluster
                      |
                      ▼
STAGE 4 (National Scale): Multi-Region Kubernetes (EKS) Federation
```

11.2 Auto Scaling Policies

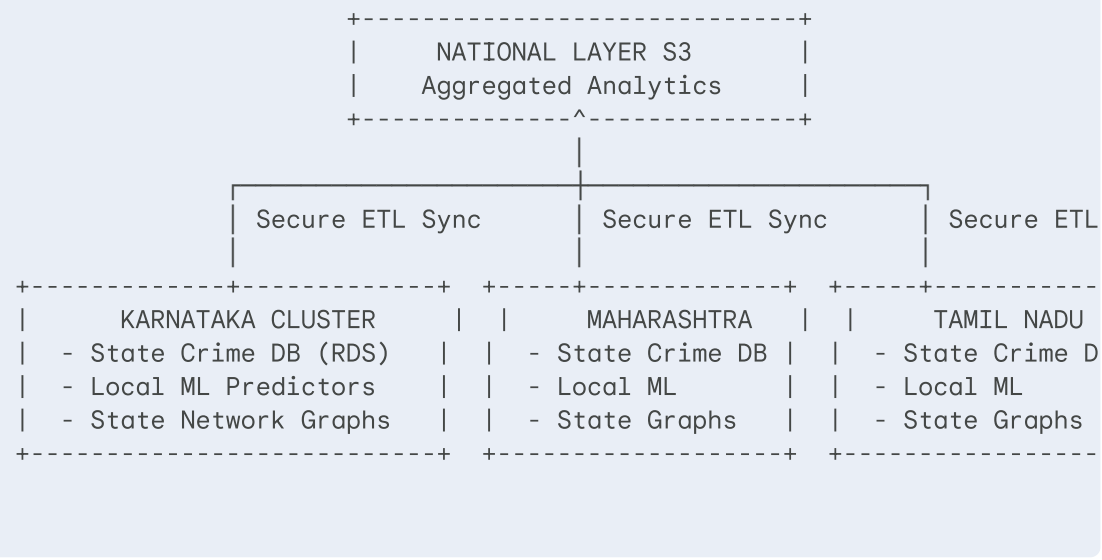
```
[System CPU / Memory Load Spikes]
               |
               ▼ (Exceeds Threshold Metric)
[Horizontal Pod Autoscaler (HPA) Triggered]
               |
               ▼ (Instructs Replica Increase)
[Kubernetes Pod Count Spawns: Pod (N) -> Pod (N+X)]
               |
               ▼ (Matches Traffic Demands)
[System Load Normalizes < Baseline Target]
               |
               ▼ (Cooldown Period Completed)
[Autonomous Pod Scale-Down Executed]
```

Scale Directive Trigger	Threshold Parameter	Autonomous Action Response

	minutes	container pods.
Memory Allocation Spike	Average pod usage > 75%	Provisions additional application replicas.
Latency Threshold	API transaction response times > 200ms	Dynamically scales container instances.

12. Multi-State Expansion Architecture

To expand LUMEN nationwide, a single centralized database is impossible due to state-level data ownership and strict legal boundaries. We utilize a **Federated Database Cluster** with distributed state-level data sovereignty.



State-Level Cluster Ownership

- Each state hosts its own isolated cloud partition containing regional FIR data, criminal identification details, and operational analytics.
- No state can directly execute master database writes to another state's infrastructure.

National Aggregation Hub

- A secure data pipeline extracts anonymized metadata into a unified national analytical storage system.
- **National Graph System:** Cross-state criminal networks are linked using anonymized hashes of suspect names, vehicle markers, and contact parameters. This maps inter-state gang structures without violating state-level data ownership laws.

13. Capacity Planning

Stage 1 — Hackathon (MVP)

- **Active User Base:** 20 Concurrent Analysts
- **Raw Database Footprint:** ~100,000 incident rows
- **Storage Allotment:** 1 Gigabyte Data Store / 5 Gigabytes S3 Asset Cache

Stage 2 — District Pilot

- **Active User Base:** 500 Concurrent Station Heads & Investigators
- **Raw Database Footprint:** ~1 Million record rows (5 Districts)
- **Storage Allotment:** 20 Gigabytes Data Store / 100 Gigabytes Asset Vault
- **Compute Baseline Allocation:** 4 AWS ECS vCPU Cores / 8 Gigabytes RAM

Stage 3 — Karnataka State Scale

- **Active User Base:** 10,000 Concurrent System Operators
- **Raw Database Footprint:** ~50 Million historical cases (All Districts)
- **Storage Allotment:** 500 Gigabytes Multi-AZ Databases / 5 Terabytes Case Assets
- **Compute Baseline Allocation:** 32 AWS ECS vCPU Cores / 128 Gigabytes Scaled RAM

Stage 4 — National Scale

- **Active User Base:** 100,000+ Active System Operators
- **Raw Database Footprint:** ~500 Million case history units (All States)
- **Storage Allotment:** 5 Terabytes Distributed Cluster Database / 100 Terabytes Document Vaults
- **Compute Baseline Allocation:** Distributed Kubernetes infrastructure across active regions (200+ Node Cores)

14. Infrastructure Cost Modeling

Scenario A — Hackathon MVP (Zoho Catalyst Deployment)

```
+-------------------------------------------------------------+
| COST PROFILE SCENARIO A: HACKATHON MVP (ZOHO CATALYST)      |
| - React UI Asset Engine Hosting:                  ₹0        |
| - Serverless Advanced API Backend:                ₹0        |
| - Relational Master PostgreSQL Storage:           ₹0        |
| - Integrated Logging, Encryption, IAM:           ₹0        |
| - Free Developer Sandbox Tier Deductions:         ₹0        |
|-------------------------------------------------------------|
| CUMULATIVE MVP RUNTIME COST:                 ₹0 / Month     |
+-------------------------------------------------------------+
```

Scenario B — District Pilot (AWS Dedicated Setup)

AWS Operational Component	Configuration Profiles	Estimated Monthly Cost
Fargate Containers	2 x Stateless API Worker Containers (2 vCPU, 4GB RAM)	₹4,000

	(t3.medium)	
File Storage Bucket	100 Gigabytes Amazon S3 Data Buckets	₹500
System Visibility	Standard AWS CloudWatch Alerts & Audits	₹500
Cumulative Cost	**Estimated Pilot Base Monthly Allocation**	**₹8,000 - ₹10,000**

Scenario C — Karnataka State Scale (AWS Production Setup)

AWS Operational Component	Configuration Profiles	Estimated Monthly Cost
Stateless Compute	8 x Auto-Scaling Fargate Backend Containers (4vCPU, 16GB)	₹35,000
Database Core	RDS PostgreSQL (db.m5.xlarge Master + Multi-AZ Read Replica)	₹25,000
Distributed Caching	AWS Elasticache Redis Node Clusters (cache.m5.large)	₹5,000
Document Vault	5 Terabytes S3 High Availability Vault Storage	₹3,000
Network & Auditing	CloudWatch Metrics, Security Hub, KMS Keys, IAM Guard	₹5,000
Cumulative Cost	**Estimated Statewide Monthly Operational Budget**	**₹70,000 - ₹90,000**

Scenario D — National Scaled Cluster (Enterprise Federation Setup)

Kubernetes Federation Component	Configuration Profiles	Estimated Monthly Cost
EKS Kubernetes Nodes	Autoscaling worker pods across multiple regions (m5.2xlarge Nodes)	₹1,20,000
Database Cluster	Global Multi-Region database (CockroachDB / Aurora Global)	₹80,000
In-Memory Caching	Distributed Redis Cluster across regional clusters	₹25,000
Distributed File System	100 Terabytes global S3 bucket with replication rules	₹20,000

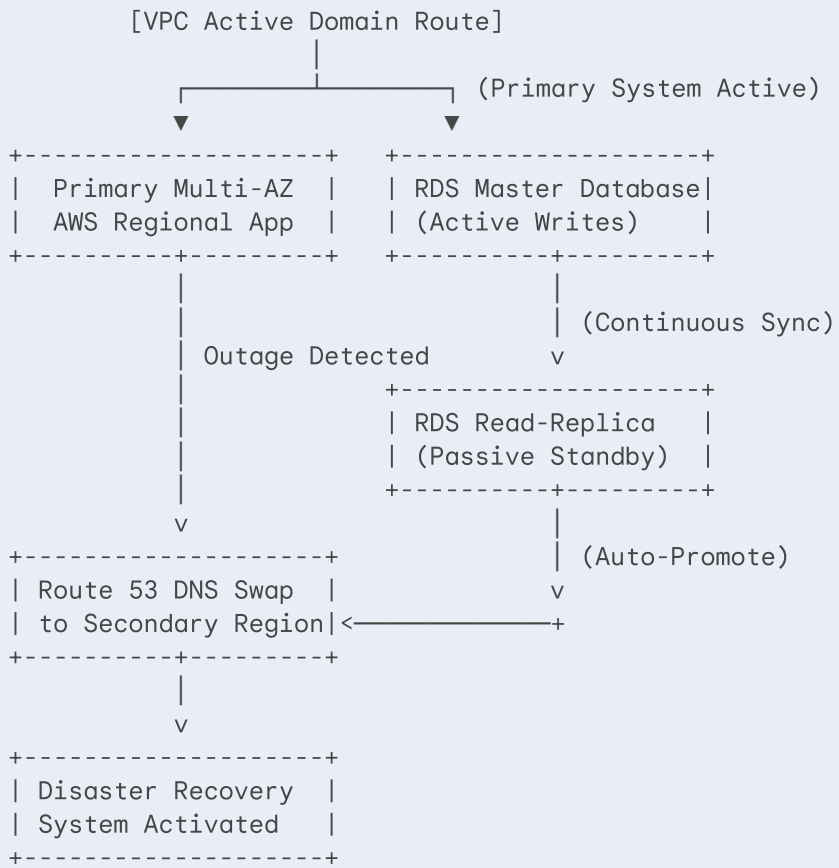
	audit ingestion	
Recovery Backups	Offsite geographical database replication	₹15,000
Cumulative Cost	**Estimated Nationwide Monthly Operational Budget**	**₹2,80,000**

15. Disaster Recovery Architecture

15.1 Backup Strategy

- **Automated RDS Backups:** Multi-AZ Amazon RDS performs daily snapshots retained for 30 days, alongside transaction logs backed up every 5 minutes.
- **Cross-Region Replication:** Database snapshots are automatically copied to a secondary AWS region to survive primary data center outages.
- **Object Vault Backups:** Amazon S3 buckets utilize Versioning alongside Cross-Region Replication (CRR) to guarantee file recovery.

15.2 Failover Strategy



15.3 Disaster Recovery Targets

Metric	Business Definition	Target Threshold

Objective (RPO)	disaster event.	sync cycles)
Recovery Time Objective (RTO)	Maximum time allowed to fully restore system availability.	**< 30 Minutes** (Executed via automated DNS failover)

16. Security Architecture

16.1 System Authentication

- **MVP Phase:** Role-selector dropdown system for rapid access during judge demonstrations.
- **Production Phase:** Strict Role-Based Access Control (RBAC) integrated with Active Directory / SAML Single Sign-On (SSO) systems.

16.2 Fine-Grained User Role Matrix

Designated Role Profile	System Access Permissions	Audit Constraints
Super Administrator	Full administrative rights, configurations, and user creation.	Global auditing tracking logs.
State Crime Analyst	Access to all district analytical views, ML forecasting models, and relationship network graphs.	No raw suspect case mutation permissions.
Investigation Officer	Access to active and historic case profiles, witness logs, and suspect records.	Access is locked strictly to assigned cases.
District Constable / SI	Access to local dashboard views, hotspot heatmaps, and active patrol rosters.	Read-only permissions, locked to district boundaries.

16.3 Data Encryption Standard

- **Data Encryption at Rest:** Database engines utilize AES-256 cryptographic standards. S3 buckets employ SSE-KMS master service encryption keys.
- **Data Encryption in Transit:** All client-to-server transactions enforce TLS 1.3 protocol handshakes. Legacy API layers employ TLS 1.2 minimum restrictions.

16.4 Incident Audit Trails

Every operation is captured by an immutable ledger:

- User profile tracking.
- Exact API resource path queried.
- Temporal timestamp recording.
- Origin IP address validation.
- Read/Write operational flags.

16.5 Security Telemetry Monitoring

behaviors.

- **AWS Security Hub:** Active security posture scans highlighting drift away from CIS benchmarks.

16.6 Comprehensive Threat Model Matrix

Explored Attack Vectors	Structural Mitigation Implementations
Data Leak & Exfiltration	Complete database encryption (at-rest and in-transit), API payload size restriction limits, egress traffic inspection.
Privilege Escalation	Rigid API middleware validation enforcing strict RBAC checks at every single route level.
DDoS Attacks	Deployment of Amazon CloudFront alongside AWS Shield to absorb volumetric request floods at the edge.
Credential Hijacking	Multi-Factor Authentication (MFA), strict password rotators, and short-duration JSON Web Token (JWT) session expirations.
Database Intrusion	Isolation of RDS and computing layers in isolated Private Subnets with strictly bound Security Group ingress rules.
Insider Access Abuse	Implementation of immutable audit trails compiled by isolated logging systems outside administrator scopes.

VOLUME 3 — JUDGE DOCUMENTATION & DECISION ANALYSIS

17. Architecture Justifications

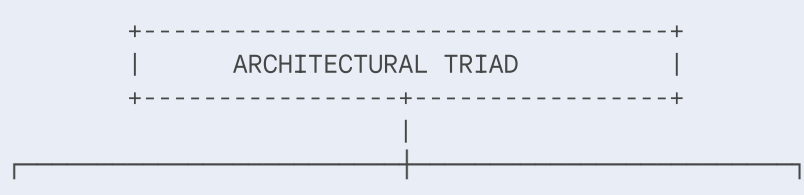
17.1 System Design Philosophy

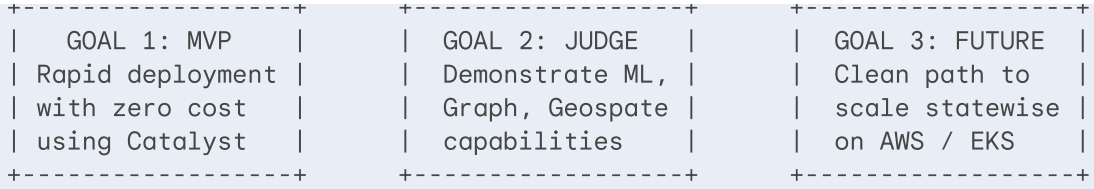
Most student hackathon projects focus on static dashboards:

```
Raw Database Storage ---> Visual Chart Panels ---> Static UI Reports
```

LUMEN is built as an active Crime Intelligence Engine. It analyzes and enriches every data record as it is ingested, turning raw inputs into dynamic, actionable intelligence.

17.2 Core Strategic Goals

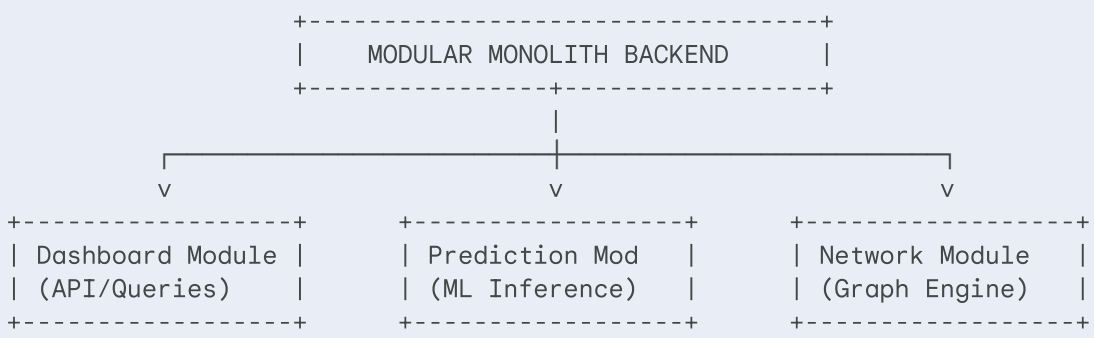




18. Architectural Decision Records (ADRs)

ADR-001: Use Modular Monolith Architecture over Microservices

- **Design Context:** Deciding the operational codebase topology for rapid development.
- **Options Evaluated:** Modular Monolith vs. Distributed Microservices.
- **Final Architectural Decision: Modular Monolith**.



Selection Rationale

- **Operational Simplicity:** Microservices introduce complex networking, service discovery, container orchestrators, and tracing middleware. For a fast-paced development sprint, this overhead directly reduces execution speed.
- **Shared Runtime Memory Execution:** Internal modules share direct variable state pathways and database connections, preventing latency bottlenecks.
- **Deterministic Deployment:** Enables deployment of a single target package to Zoho Catalyst with confidence.

ADR-002: Use PostgreSQL as the Primary Core Relational Engine

- **Design Context:** Selection of primary analytical storage tier.
- **Options Evaluated:** PostgreSQL vs. MongoDB vs. Apache Cassandra.
- **Final Architectural Decision: PostgreSQL**.

Selection Rationale

- **Highly Relational Schema:** Crime records represent an intensely interconnected data map (Criminals, Victims, Locations, Incidents, and Vehicles). Storing this in an unstructured NoSQL database like MongoDB requires application-level join logic, reducing database optimization opportunities.

aggregates, and transactional workloads efficiently.

- **Scale-Up Path:** Supports seamless transition to AWS RDS and spatial indexing tools.

ADR-003: Use FastAPI Backend over Node.js

- **Design Context:** Selecting the primary server-side API framework.
- **Options Evaluated:** FastAPI (Python) vs. Node.js (Express) vs. Spring Boot (Java).
- **Final Architectural Decision: FastAPI**.

Selection Rationale

- **Native AI/ML Ecosystem Integration:** Our analytics core relies on scientific Python libraries (`scikit-learn`, `networkx`, `pandas`). Building the API layer in Node.js would require inter-process communication overhead to trigger Python execution scripts:

```
Client Request ---> Express API (NodeJS) ---> Child Process Call ---> Python M
```

FastAPI runs natively on Python, allowing direct memory access to machine learning models:

```
Client Request ---> FastAPI API ---> Direct Python ML Inference Memory Call
```

- **Performance:** Built on Starlette and ASGI, FastAPI performance approaches Node.js while keeping code clean and readable.
- **Automatic OpenAPI Specs:** FastAPI automatically builds interactive Swagger documentation, allowing judges to test raw endpoints instantly.

ADR-004: Use CPU-Based ML Models over Deep Learning

- **Design Context:** Selecting the primary predictive modeling approach.
- **Options Evaluated:** CPU-Based Machine Learning (Scikit-Learn, Random Forests) vs. Deep Learning Models (PyTorch, GPUs).
- **Final Architectural Decision: CPU-Based Machine Learning**.

Selection Rationale

- **Resource Optimization:** Deep learning models require specialized GPU clusters, which increases operational budgets.
- **Data Scale:** State-level spatiotemporal crime datasets are processed effectively by tree-based machine learning classifiers.
- **Explainable AI (XAI) Compliance:** Random Forest architectures provide transparent feature importance matrices. This lets police officers verify the exact real-world attributes driving risk scores, ensuring trust in the system's output.

19. Tradeoff Analysis

Codebase Organization: Monolith vs. Microservices

Evaluated	Modular Monolith (LUMEN	Distributed Microservices

Metric	Choice)	
Deployment Speed	**High** (Single target deployment)	**Low** (Requires configuring multiple deployment pipelines)
Setup Complexity	**Low** (Single repository setup)	**High** (Requires service mesh, ingress, and gateway orchestration)
Operational Costs	**Low** (Consolidated container instances)	**High** (Baseline container runtime expenses per service)
System Debugging	**Simple** (Traceable inside a single debugger console)	**Hard** (Requires distributed tracing tools like Jaeger)
Scaling Granularity	**Moderate** (Entire monolith scales together)	**Excellent** (Can scale individual API endpoints independently)

Primary Database: PostgreSQL vs. MongoDB

Evaluated Metric	PostgreSQL (LUMEN Choice)	MongoDB
Join Queries	**Excellent** (Native relational schema)	**Poor** (Requires denormalizing data or application-level merges)
Transactional Integrity	**Strong** (Full ACID compliance)	**Moderate** (Document-level transactional capabilities)
Aggregation Complexities	**Excellent** (Advanced CTE and analytical window queries)	**Moderate** (Limited aggregation framework capabilities)
Dynamic Schemas	**Moderate** (Requires database migration workflows)	**High** (Schemaless document-level structures)

Container System: ECS vs. Kubernetes (EKS)

Evaluated Metric	ECS on AWS Fargate (LUMEN Pilot)	Kubernetes (EKS)
Infrastructure Setup	**Easy** (100% managed serverless containers)	**Hard** (Complex master node and cluster configurations)
Operational Overhead	**Low** (AWS automates all host VM maintenance)	**High** (Requires manual cluster upgrades and Helm management)
Minimum Base Cost	**Lower** (Billed purely by active execution seconds)	**Higher** (Requires persistent running master control instances)

Power	lifecycles)	automated healing)

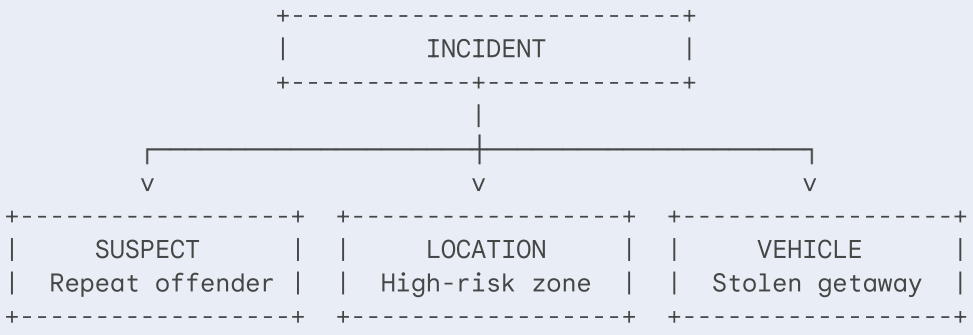
Hosting Base: Zoho Catalyst vs. AWS

Evaluated Metric	Zoho Catalyst (LUMEN MVP)	Amazon Web Services (AWS)
Time to Market	**Instant** (Direct deployment commands out-of-the-box)	**Moderate** (Requires terraform configuration and networks)
Base MVP Cost	**Near Zero** (Free developer sandbox thresholds)	**Higher** (Immediate base virtual private cloud costs)
Learning Curve	**Low** (Simple web administrative layouts)	**High** (Demands professional certified architecture knowledge)
Infrastructure Control	**Moderate** (Abstracted serverless platform)	**Complete** (Full bare-metal structural configuration control)

20. Why PostgreSQL?

20.1 Relational Integrity of Crime Data

Crime data is intrinsically relational. Incidents do not exist in isolation; they are connected to suspect profiles, location indexes, witness accounts, and stolen physical assets.



These complex real-world connections are handled naturally by PostgreSQL through:

- **Relational Joins:** Instantly links suspects to crime locations using relational foreign key indexing.
- **ACID Transactions:** Ensures evidence trails are recorded with complete transactional integrity, preventing duplicate or corrupted records.

20.2 Built-In Analytical Tools

PostgreSQL features advanced processing engines like:

- **Window Functions:** Processes rolling averages, spatial trends, and crime spike metrics in a single query pass.
- **Common Table Expressions (CTEs):** Simplifies complex multi-step queries into readable, performant logic blocks.

PostgreSQL easily adapts as LUMEN scales:

- **PostGIS Extension:** Integrates geospatial analytics, enabling fast coordinate indexing and polygon calculations without third-party services.
- **Partitioning & Replication:** Easily scales to handle millions of statewide records without database engine migration.

21. Why FastAPI?

21.1 Native ML Performance

Our machine learning classifiers are written natively in Python. Running FastAPI eliminates the translation layers required by Node.js or Java, enabling direct memory integration between the API routing layer and predictive models.

21.2 High Concurrency

FastAPI leverages Python's `async/await` ecosystem built on top of Starlette and Uvicorn. This enables high concurrency rates that approach Node.js, making the platform responsive under heavy user loads.

21.3 Automatic OpenAPI Integration

FastAPI automatically builds interactive Swagger and ReDoc documentation:

```
+-------------------------------------------------------+
| LUMEN API SWAGGER UI                                  |
|                                                       |
|   GET  /api/dashboard     [Try it out]  ---> Active UI |
|   GET  /api/predictions   [Try it out]  ---> Active UI |
|   GET  /api/network       [Try it out]  ---> Active UI |
+-------------------------------------------------------+
```

This self-documenting design enables rapid client-server integration and allows hackathon judges to verify API execution instantly.

22. Why Graph Analytics?

22.1 Limitations of Relational Databases

Standard relational databases answer categorical questions efficiently:

- *How many burglaries occurred in Bengaluru last month?*
- *Which suspects are currently released on bail?*

However, they struggle with relationship-driven analysis:

- *Who is the central coordinator of this multi-district burglary syndicate?*
- *Are there hidden relationships between seemingly disconnected crime families?*

22.2 Network Topologies for Law Enforcement

```
A (Leader Node) ──────────────── B (Node)
      |                              |
      |                              |
      v                              v
C (Broker Node) ──────────────── D (Node)
```

Graph analytics uncovers hidden networks through:

- **Degree Centrality:** Identifies highly connected individuals with the most immediate relationships.
- **Betweenness Centrality:** Highlights "broker nodes" who act as critical information paths and bridge distinct criminal cells.
- **Community Detection (Girvan-Newman):** Groups loosely associated suspects into clear, organized syndicate cells, mapping gangs automatically.

23. Explainable AI Strategy

23.1 The Black-Box Prediction Problem

If an AI platform issues a danger alert saying:

```
District Warning Risk Score = 87%
```

Without explanation, commanders cannot confidently deploy patrol units. **Black-box predictions undermine operational trust.**

23.2 Explainable Predictions (XAI)

Every prediction generated by LUMEN includes three descriptive context parameters:

1. Prediction Confidence Metric

- **Output Example:** `Confidence: 83%`

2. Key Influencing Variables

- **Output Example:**
  - Historic Crime Density: `+35%` (Weight)
  - Spatiotemporal Seasonality Factor: `+20%` (Weight)
  - Socio-Economic Vulnerability Index: `+15%` (Weight)

3. Human-Readable Operational Summaries

- **Output Example:**

  > *"This district is classified as high-risk due to repeated burglary activity during the previous three months combined with increasing crime density."*

24. Future Scope

```
+-------------------------------------------------------------+
| Phase 5 — ADVANCED AI INTEGRATION                           |
| Automatic NLP FIR extraction, voice query, crime timeline   |
+-------------------------------------------------------------+
| Phase 4 — NATIONAL FEDERATED INTELLIGENCE                   |
| Cross-state analytics, cross-border gang tracking network   |
+-------------------------------------------------------------+
| Phase 3 — STATEWIDE ROLLOUT                                 |
| Full scale state police integration, multi-user systems     |
+-------------------------------------------------------------+
| Phase 2 — DISTRICT PILOT                                    |
| 5-District deployments, live station reporting workflows     |
+-------------------------------------------------------------+
| Phase 1 — CURRENT MVP DELIVERABLE                           |
| Serverless React app, FastAPI backend, core ML analytics    |
+-------------------------------------------------------------+
```

25. Risk Assessment

Potential Risk Identified	Risk Level	Structural Mitigation Strategy
Dirty / Malformed Ingested Data	**High**	Introduce strict backend validation pipelines and data-cleansing middleware before storage.
Poor Predictive Accuracy	**Medium**	Continuously evaluate models against real-world occurrences to dynamically adjust feature weights.
API Response Bottlenecks	**Medium**	Deploy in-memory Redis caching to cache high-frequency read queries.
Rapid Database Growth	**High**	Implement native database partitioning, automated archiving policies, and horizontal scaling.
Feature Scope Creep	**High**	Prioritize the MVP feature set to deliver a stable, reliable core platform on time.
System Integration Obstacles	**High**	Define and enforce strict API contracts (Pydantic schemas) early to decouple front-end and back-end work.

26. Technical Debt Register

MVP Tradeoffs for Hackathon Speed

Incurred Debt	Architectural Compromise	Permanent Future Resolution
Single Database Instance	Shared analytical and API storage to avoid database sync latency during development.	Implement Read-Replicas to isolate API read requests from resource-heavy ML processes.

Backend	gateways inside a single runtime to speed up development.	independent serverless container workers.
Batch Analytical Model Run	Calculations run as scheduled batch scripts rather than real-time continuous pipelines.	Implement event-driven message queues (Celery/RabbitMQ) for real-time model updates.
Simplified Credentials Guard	Role-selector mockup for fast demonstrations during judging rounds.	Deploy professional OAuth2/OIDC protocols integrated with Active Directory.

27. Presentation Talking Points

Core Talking Points

1. **The Core Problem:** Legacy police systems are static storage vaults. They store data but **do not generate intelligence**.
2. **The LUMEN Solution:** LUMEN acts as an active intelligence engine, transforming raw files into Predictions, Relationship Graphs, and Anomaly alerts.
3. **The Key Innovation:** The system shifts policing from **Reactive** to **Predictive**.
4. **Our Technical Edge:** Full integration of spatial clustering (DBSCAN), network centrality, and anomaly isolation (Isolation Forest) into a single, high-performance platform.
5. **The Real-World Roadmap:** Designed with a clear, step-by-step path to scale from a zero-cost hackathon prototype to a resilient, enterprise-grade cloud architecture.

28. Judge Q&A Cheat Sheet

Q: Why did you choose PostgreSQL over MongoDB?

- **A:** Crime datasets are highly relational (criminals, victims, locations, incidents). PostgreSQL handles these connected models out-of-the-box, ensuring full transaction safety (ACID) and robust relational querying that NoSQL engines cannot match natively.

Q: Why choose Scikit-Learn over Deep Learning neural networks?

- **A:** Crime forecasting requires highly explainable models. Neural networks operate as "black boxes," whereas Random Forests provide transparent feature importance rankings. Additionally, CPU-based models deploy with near-zero infrastructure costs while matching performance.

Q: Why deploy a Monolith instead of Microservices?

- **A:** For a hackathon sprint, microservices introduce unnecessary operational complexity and infrastructure overhead. A modular monolith provides the benefits of rapid compilation and fast development while maintaining clean module boundaries for future service separation.

Q: How does this scale to a national level?

- **A:** We employ a Federated Architecture. Each state maintains full ownership over its regional data cluster to comply with privacy laws, while an encrypted, metadata-only ETL pipeline syncs anonymized indicators to a national layer to map inter-state gang networks.

- **A:** Fusing four key analytical dimensions—**Predictive Forecasting, Network Analytics, Geographic Mapping, and Anomaly Detection**—into a single, unified law-enforcement platform.